ArcadeHaven - DFDs

Owner: Diogo Sousa
Reviewer: Diogo Sousa
Contributors: Diogo Pereira
Date Generated: Sun Apr 19 2026

Executive Summary

High level system description

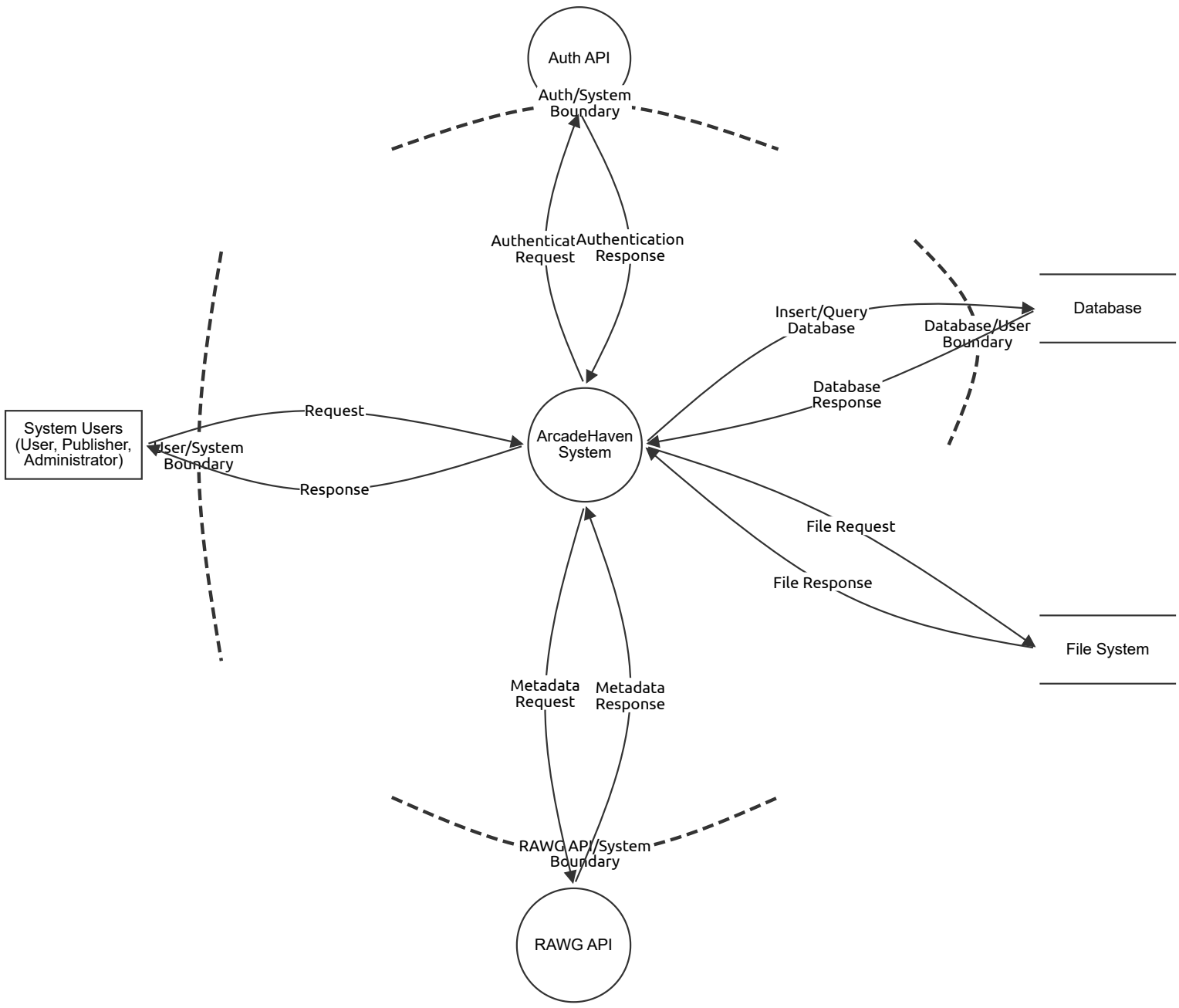
Dataflow diagrams of the ArcadeHaven system

Summary

Total Threats	123
Total Mitigated	0
Total Open	123
Open / Critical Severity	16
Open / High Severity	42
Open / Medium Severity	65
Open / Low Severity	0

DFD - Level 0

ArcadeHaven DFD - Level 0



DFD - Level 0

System Users (User, Publisher, Administrator) (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

ArcadeHaven System (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Auth API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

RAWG API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Database (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

File System (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Authentication Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Authentication Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Insert/Query Database (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

File Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

File Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Metadata Request (Data Flow)

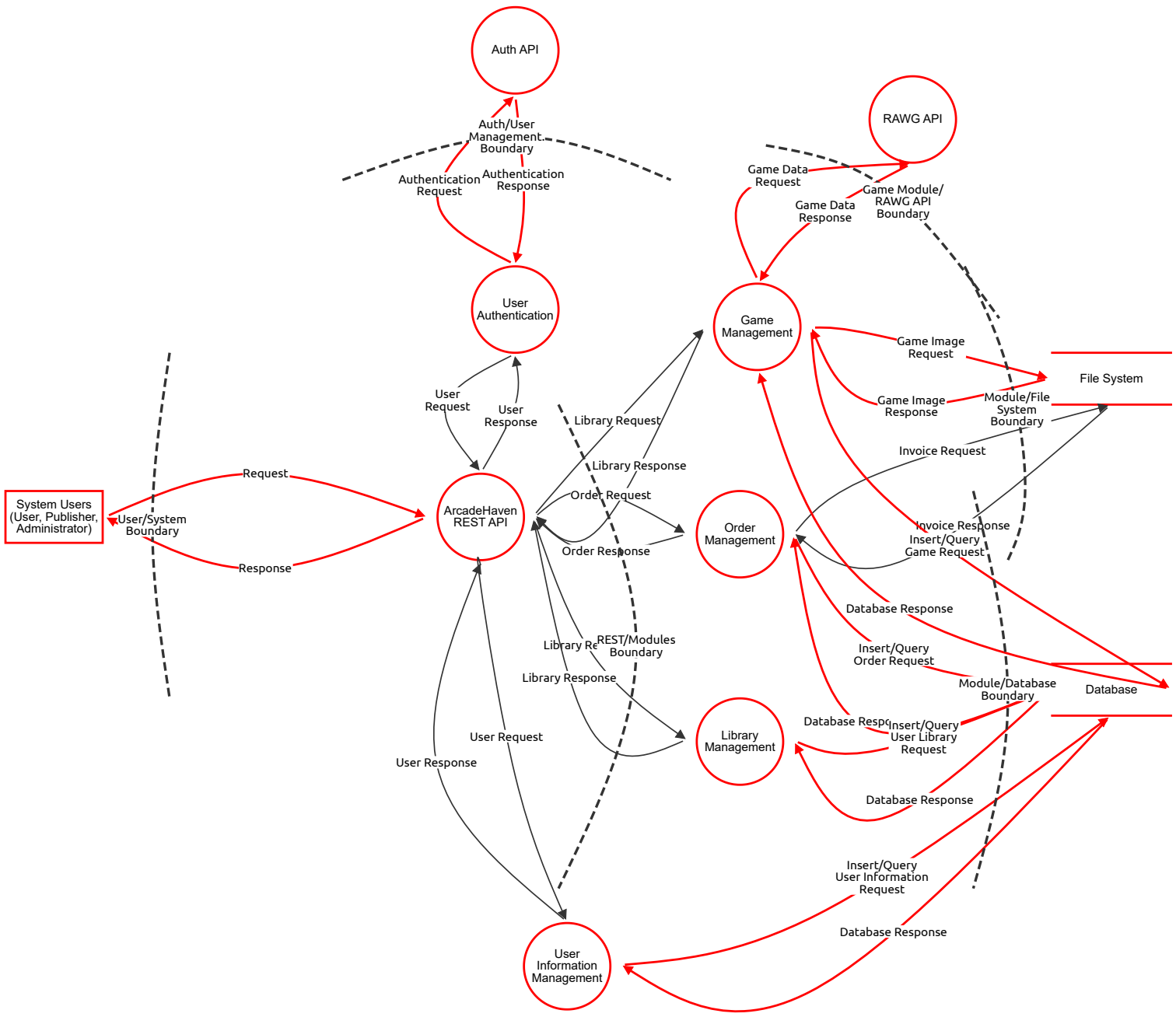
Number	Title	Type	Severity	Status	Score	Description	Mitigations

Metadata Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

DFD - Level 1 - ArcadeHaven System

ArcadeHaven system module DFD



DFD - Level 1 - ArcadeHaven System

System Users (User, Publisher, Administrator) (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
47	User identity impersonation	Spoofing	Medium	Open		Any unauthenticated or weakly-authenticated actor can claim to be a legitimate user, publisher, or admin at the trust boundary.	An attacker reuses stolen credentials to log in as a Publisher and approve/modify game listings. An attacker forges a JWT sub claim to act as Admin.
48	Denial of performed actions	Repudiation	Medium	Open		Without audit logging, a user can deny having purchased a game, a publisher can deny having uploaded malicious content, an admin can deny having approved a game.	A Publisher uploads a game with malicious content, then claims "I never uploaded that file." No log exists to tie the upload to their identity and timestamp.

ArcadeHaven REST API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
8	Request body manipulation / mass assignment	Tampering	Medium	Open		Buyer sends a POST /orders with a modified price field, paying less for a game if the API trusts client-submitted prices instead of looking up the canonical price from the DB.	Never trust client-submitted prices; resolve price server-side from the Games datastore; use strict DTO validation with @Valid; disable mass assignment via explicit @JsonProperty mapping.
9	JWT not validated on every route	Spoofing	Critical	Open		Attacker crafts a request with an unsigned or expired token to a Spring endpoint that lacks @PreAuthorize; gains access as any user.	Global JWT validation filter in Spring Security; no endpoint bypasses; token expiry enforced; alg:none attack prevention (reject tokens with no algorithm).
10	Insufficient request logging	Repudiation	High	Open		An admin deletes a user account. No structured log exists with the admin's identity, timestamp, and target. They later deny the action.	Structured access log for all mutating operations (POST/PUT/PATCH/DELETE) including authenticated user ID, resource path, body hash, and timestamp; send logs to a tamper-evident SIEM.
12	Verbose error messages expose internals	Information disclosure	Critical	Open		A 500 error leaks a stack trace containing table names, ORM query structure, or DB credentials to the caller.	Global exception handler returning generic error responses; suppress stack traces in production; use structured error codes; never include DB exception messages in HTTP responses.
13	No rate limiting on public endpoints	Denial of service	Medium	Open		Attacker floods POST /orders or GET /games with thousands of requests/second, exhausting the Spring Boot thread pool and making the API unavailable to legitimate users.	API gateway or Spring rate-limiting filter (e.g. Bucket4j); limit per IP and per authenticated user; return 429 with Retry-After; circuit breaker pattern for downstream DB calls.
14	Missing role checks on admin routes	Elevation of privilege	Critical	Open		A Buyer calls DELETE /admin/games/{id} and it succeeds because the endpoint validates only authentication (valid JWT) but not the required ADMIN role in the Keycloak token claims.	Role-based access control enforced at the method level via @PreAuthorize("hasRole('ADMIN')"); integration tests for every protected endpoint verifying that Buyer/Publisher tokens are rejected; automated security scanning in CI pipeline with SonarQube.

User Authentication (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
15	Credential brute-force / credential stuffing	Spoofing	High	Open		Attacker tries thousands of email/password combinations from a leaked credential list against the Keycloak login endpoint to hijack accounts.	Account lockout after N failed attempts in Keycloak; CAPTCHA on login; MFA for all roles; breached-password detection; monitor for anomalous login patterns.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
16	JWT claims manipulation	Tampering	High	Open		If JWT signing key is weak or exposed, an attacker crafts a token elevating their role from BUYER to ADMIN in the realm_access.roles claim.	Use RS256 asymmetric signing in Keycloak; rotate signing keys regularly; back-end must verify signature against Keycloak's JWKS endpoint, not just parse the payload; never trust client-modified tokens.
17	No authentication event audit trail	Repudiation	High	Open		A compromised admin account performs bulk deletions. Without login/logout and token-issue audit logs, there is no proof of when the account was used or from which IP.	Enable Keycloak event logging (login, logout, token issue, failed login); forward events to a centralised log; retain logs for at least 90 days; alert on suspicious patterns (login from new geo, many failures).
18	Token leakage via insecure storage or logging	Information disclosure	High	Open		JWT access tokens are logged in Spring Boot request logs at DEBUG level. A developer with log access can impersonate any user whose token appears in the log.	Never log full Authorization headers; mask tokens in all log output; enforce HTTPS-only token transport; set SameSite=Strict and Secure on any auth cookies; use short token lifetimes.
19	Keycloak token endpoint flooded	Denial of service	High	Open		Attacker hammers POST /realms/arcadehaven/protocol/openid-connect/token with invalid credentials, locking out legitimate users or exhausting Keycloak's DB connection pool.	Rate-limit the Keycloak token endpoint at the reverse proxy / WAF layer; deploy Keycloak in HA mode with a dedicated DB; monitor token endpoint latency as a health signal.
20	Role assignment via Keycloak admin console exposure	Elevation of privilege	Critical	Open		Keycloak admin console is accessible on the public internet. An attacker exploits default credentials or an unpatched CVE to assign ADMIN role to their account.	Restrict Keycloak admin console to internal network / VPN only; change default admin credentials immediately; keep Keycloak patched; enable Keycloak audit log for all realm admin operations.

Auth API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
51	Accepting tokens from untrusted issuers	Spoofing	High	Open		Spring Security misconfigured to accept any JWT, allowing an attacker's self-issued token to authenticate as any user.	Whitelist exactly one issuer URI in Spring Security config (spring.security.oauth2.resourceserver.jwt.issuer-uri).
52	Token claims not re-validated after role change	Tampering	Medium	Open		A user's Keycloak role is downgraded by an admin, but their existing JWT (valid for 15 more minutes) still grants the old role because claims aren't re-checked against Keycloak on each request.	Implement token introspection or keep token TTL very short (≤5 min) for sensitive role changes; maintain a token revocation list or use Keycloak logout events.
53	No auth decision logging in Spring layer	Repudiation	Medium	Open		Spring Security rejects a request for insufficient privileges but logs nothing; the access attempt cannot be investigated later.	Custom AuthenticationFailureHandler and AccessDeniedHandler logging rejected attempts with user ID, resource, and timestamp.
54	Token logged at DEBUG level	Information disclosure	Medium	Open		Spring Boot DEBUG logging includes full Authorization header; a developer with log access can impersonate any logged user.	Never log Authorization headers; mask tokens in all log formatters; use structured logging with explicit field whitelists.
55	Repeated JWKS fetches on every request	Denial of service	Critical	Open		JWKS endpoint called on every API request; Keycloak slowdown cascades to full API unavailability.	Cache JWKS keys locally with configurable TTL; background refresh; circuit breaker around JWKS fetch.
56	Role extracted from wrong JWT field	Elevation of privilege	High	Open		Spring Security reads roles from realm_access.roles but attacker crafts a token with roles in a custom claim that is mistakenly also trusted, granting ADMIN	Explicitly configure which JWT claim path maps to Spring authorities; reject tokens with unexpected claim structures.

Game Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
21	Publisher identity spoofing	Spoofing	High	Open		Attacker obtains a valid Publisher JWT and submits games on behalf of a legitimate publisher, polluting their catalogue with malicious titles.	Validate JWT publisher ID against the game's owning publisher in every write operation; RBAC enforcement at method level; Keycloak MFA for Publisher accounts.
22	Malicious file upload (images/game files)	Tampering	High	Open		A Publisher uploads a .exe disguised as a game screenshot. If the server only validates Content-Type header (spoofable), the file is stored as-is and served to buyers.	Server-side file type validation using magic bytes (not just extension or Content-Type); antivirus/malware scanning pipeline on all uploads (e.g. ClamAV); store uploads outside the web root; generate a new UUID filename on save; restrict executable file types.
23	Unsigned game status changes	Repudiation	Medium	Open		Admin approves a game (PENDING → ACTIVE) but no log records which admin, when, or from which IP. Admin later denies the approval after a compliance audit.	Audit log every game state transition with actor identity, timestamp, and previous/new state; store in append-only table; require admin comment on approval/rejection.
24	RAWG metadata injection into stored fields	Information disclosure	Medium	Open		RAWG API returns a game description containing a script tag. If stored unsanitised and later rendered in HTML, it becomes a stored XSS vector disclosing session tokens of visiting users.	Sanitise all RAWG-sourced fields before storage using an HTML sanitiser (e.g. OWASP Java HTML Sanitizer); apply output encoding at the frontend; enforce Content-Security-Policy.
25	Large file upload exhausting disk/memory	Denial of service	Medium	Open		Publisher (or attacker with a stolen Publisher token) uploads a 10 GB file as a "game screenshot", consuming all available disk on the file system and blocking invoice generation.	Enforce maximum file size limits at the API gateway and in the multipart upload handler; check available disk space before write; implement per-publisher upload quota; alert on disk usage thresholds.
26	Publisher self-approving games	Elevation of privilege	High	Open		If the ADMIN role check on the game approval endpoint is missing, a Publisher can call PATCH /games/{id}/approve and set their own game to ACTIVE, bypassing admin review.	Strictly separate Publisher and Admin operations in the controller layer; approval endpoint requires ADMIN role; automated integration test verifies Publisher token returns 403 on approval endpoint.

Order Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
27	Placing orders on behalf of another user	Spoofing	High	Open		Attacker intercepts a Buyer's JWT and submits an order that charges the victim's account while delivering the activation key to the attacker's library.	Bind order's buyer_id server-side from the authenticated JWT sub claim, never from the request body; validate JWT on every order endpoint; revoke compromised tokens via Keycloak session invalidation.
28	Order price or item count manipulation	Tampering	Medium	Open		Buyer modifies the POST /orders request body to set price: 0.01 for a €59.99 game. If the server processes the client-supplied price, the item is effectively free.	Always resolve item price from the Games datastore at order creation time; never accept price in the order creation payload; recalculate totals server-side; validate order items against active game catalogue.
29	Disputing completed purchases	Repudiation	Medium	Open		Buyer claims "I never bought that game." Without a signed, timestamped order record, the system cannot prove the purchase occurred with the user's authenticated session.	Immutable order records with buyer ID, timestamp, game ID, price at purchase, and invoice PDF hash; generate and store non-repudiable receipts; consider order signing with a server-side key for high-value transactions.
30	Activation key leakage in API responses	Information disclosure	Medium	Open		GET /orders/{id} returns the activation key in plain JSON. If the API response is cached by a CDN or proxy, the key is exposed to anyone with a cached response URL.	Never cache endpoints that return activation keys (Cache-Control: no-store); serve activation keys only after re-authentication confirmation; consider one-time retrieval tokens for key delivery; encrypt keys at rest in the DB.
31	Order flooding / cart abuse	Denial of service	High	Open		Attacker creates thousands of incomplete orders per second, exhausting DB connection pool and preventing legitimate checkouts. Invoice PDF generation (CPU-intensive) can be weaponised similarly.	Rate-limit order creation per authenticated user; implement async invoice generation (message queue); limit pending/abandoned orders per user; add circuit breaker around PDF generation service.
32	Accessing another user's order history	Elevation of privilege	Medium	Open		Buyer calls GET /orders?userId=2 and retrieves another user's full purchase history and activation keys because the endpoint filters only by the query param, not the JWT sub.	Filter all order queries by the authenticated user's ID from the JWT, ignoring any userId parameter in the request; add automated IDOR (Insecure Direct Object Reference) tests to the CI pipeline.

Library Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
33	Accessing another user's library	Spoofing	Medium	Open		Attacker crafts GET /library/{userId} where userId belongs to another Buyer, reading their full game collection and activation keys.	Library access enforced to authenticated user's own ID from JWT; no userId parameter accepted from client for personal library operations; strict IDOR prevention.
34	Injecting games into another user's library	Tampering	Medium	Open		An attacker with ADMIN-level access (or exploiting missing auth) calls POST /library/{victimUserId}/entries to grant games without purchase, bypassing the order flow.	Library write operations gated to Order Management completion events only (event-driven entry creation); no direct external API to add library entries; admin library modifications require extra audit trail.
35	Untracked profile changes	Repudiation	Medium	Open		A user changes their email address (used for invoice delivery) then disputes a charge, claiming "That's not my email." No audit log shows they changed it themselves.	Log all profile mutations (email, password, display name) with timestamp and actor ID; require re-authentication (current password) for sensitive profile changes; send notification email to old address on change.
36	PII exposure in API responses	Information disclosure	Medium	Open		GET /users/{id} returns full user object including hashed password, internal role flags, and email for all callers, not just the owner, due to missing @JsonIgnore on sensitive fields.	Use dedicated response DTOs that exclude internal fields; apply @JsonIgnore to sensitive fields; ensure GET /users/{id} returns only data the authenticated caller is entitled to; never return password hashes.
37	Library query returning unbounded results	Denial of service	Medium	Open		A user with 10,000 library entries calls GET /library without pagination. The ORM loads all records into memory, causing OOM error or extreme latency.	Mandatory pagination on all list endpoints (max page size enforced server-side); use @PageableDefault with a maximum page size limit; lazy-load relationships in JPA/Hibernate.
38	Horizontal privilege escalation via profile edit	Elevation of privilege	High	Open		Buyer calls PATCH /users/{adminUserId}/profile and, if the endpoint only checks for authentication rather than ownership, modifies an admin's email or display name.	All user profile updates must verify that the authenticated user's ID matches the target resource owner; admins managing other accounts require explicit ADMIN role check; prevent users from modifying their own roles.

User Information Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
33	Accessing another user's library	Spoofing	Medium	Open		Attacker crafts GET /library/{userId} where userId belongs to another Buyer, reading their full game collection and activation keys.	Library access enforced to authenticated user's own ID from JWT; no userId parameter accepted from client for personal library operations; strict IDOR prevention.
34	Injecting games into another user's library	Tampering	Medium	Open		An attacker with ADMIN-level access (or exploiting missing auth) calls POST /library/{victimUserId}/entries to grant games without purchase, bypassing the order flow.	Library write operations gated to Order Management completion events only (event-driven entry creation); no direct external API to add library entries; admin library modifications require extra audit trail.
35	Untracked profile changes	Repudiation	Medium	Open		A user changes their email address (used for invoice delivery) then disputes a charge, claiming "That's not my email." No audit log shows they changed it themselves.	Log all profile mutations (email, password, display name) with timestamp and actor ID; require re-authentication (current password) for sensitive profile changes; send notification email to old address on change.
36	PII exposure in API responses	Information disclosure	Medium	Open		GET /users/{id} returns full user object including hashed password, internal role flags, and email for all callers, not just the owner, due to missing @JsonIgnore on sensitive fields.	Use dedicated response DTOs that exclude internal fields; apply @JsonIgnore to sensitive fields; ensure GET /users/{id} returns only data the authenticated caller is entitled to; never return password hashes.
37	Library query returning unbounded results	Denial of service	Medium	Open		A user with 10,000 library entries calls GET /library without pagination. The ORM loads all records into memory, causing OOM error or extreme latency.	Mandatory pagination on all list endpoints (max page size enforced server-side); use @PageableDefault with a maximum page size limit; lazy-load relationships in JPA/Hibernate.
38	Horizontal privilege escalation via profile edit	Elevation of privilege	High	Open		Buyer calls PATCH /users/{adminUserId}/profile and, if the endpoint only checks for authentication rather than ownership, modifies an admin's email or display name.	All user profile updates must verify that the authenticated user's ID matches the target resource owner; admins managing other accounts require explicit ADMIN role check; prevent users from modifying their own roles.

RAWG API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
49	Rogue/spoofed RAWG API endpoint	Spoofing	High	Open		The system calls an external REST API. DNS poisoning or a supply-chain compromise could redirect calls to a malicious server returning crafted metadata.	Attacker performs DNS cache poisoning so rawg.io resolves to their server, which returns forged game metadata (e.g. injecting malicious URLs as game image links) that gets stored in the database.
50	No accountability for bad external data	Repudiation	Medium	Open		There is no mechanism to prove which version of RAWG data was consumed at a given time, making it impossible to attribute data-quality incidents.	RAWG returns incorrect age-rating metadata; a minor accesses an adult game. There is no record of what was returned at the time of ingestion, making root-cause analysis impossible.

File System (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
43	Path traversal / arbitrary file write	Tampering	High	Open		A Publisher submits a filename of ../../etc/cron.d/backdoor as the game image filename. If the application uses the client-supplied filename directly, it writes the file outside the intended upload directory.	Always generate a server-side UUID filename; never use client-supplied filenames in file system paths; resolve the canonical path and verify it is within the intended base directory before writing; run the application with minimal OS file-system permissions.
44	No file access/modification audit trail	Repudiation	High	Open		An invoice PDF file is modified or deleted. Without file-system access logging, it is impossible to determine who accessed or changed it, violating financial audit requirements.	Log all file read/write/delete operations at the application layer with actor ID and timestamp; use OS-level file integrity monitoring (e.g. auditd on Linux) for the invoices directory; consider WORM (write-once-read-many) storage for invoice PDFs.
45	Direct file access bypassing application auth	Information disclosure	High	Open		Invoice PDFs and activation key files are stored in a directory served directly by the web server (e.g. Nginx static file serving). Guessing the file path allows unauthenticated download of any invoice.	Store sensitive files (invoices, activation key files) outside the web-serving root; serve them only via authenticated application endpoints that verify the requester owns the resource; generate time-limited signed download URLs for file delivery.
46	Disk exhaustion via upload abuse	Denial of service	Medium	Open		An attacker with a valid Publisher token uploads hundreds of large image files in rapid succession, filling the disk partition and preventing invoice PDF generation, which requires disk writes to complete orders.	Enforce per-user and global upload size quotas; monitor disk usage with automated alerts at 70%/85%/95% thresholds; store uploads on a separate partition or object storage (e.g. S3) isolated from application and invoice storage; implement async file processing with backpressure.

Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
60	MitM — request/response body modification	Tampering	High	Open		An attacker on the network path intercepts an HTTP (non-TLS) order creation request and modifies the game_id or quantity before it reaches the server.	Enforce HTTPS (TLS 1.2+) for all traffic; HTTP Strict Transport Security (HSTS) header; disable HTTP listener entirely in production; use certificate pinning in the frontend if applicable.
61	Sensitive data in HTTP responses over unencrypted channel	Information disclosure	Medium	Open		Activation keys, invoice data, and PII returned in API responses are transmitted in cleartext over HTTP, readable by any network observer.	TLS everywhere; mark all sensitive response fields as non-cacheable (Cache-Control: no-store, no-cache); use secure, HttpOnly, SameSite cookies for any session identifiers; avoid putting sensitive data in URL query parameters (logged by proxies and servers).
62	HTTP flood / slow-loris attack	Denial of service	High	Open		Attacker opens thousands of slow HTTP connections to the Spring Boot server, holding threads open without completing requests, exhausting the server thread pool and blocking legitimate traffic.	Deploy behind a reverse proxy (Nginx) that handles connection timeouts; set maximum request body size and read timeout; use a WAF to detect and block HTTP flood patterns; Nginx's limit_conn and limit_req modules; configure Spring Boot's server.connection-timeout.

Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
60	MitM — request/response body modification	Tampering	High	Open		An attacker on the network path intercepts an HTTP (non-TLS) order creation request and modifies the game_id or quantity before it reaches the server.	Enforce HTTPS (TLS 1.2+) for all traffic; HTTP Strict Transport Security (HSTS) header; disable HTTP listener entirely in production; use certificate pinning in the frontend if applicable.
61	Sensitive data in HTTP responses over unencrypted channel	Information disclosure	Medium	Open		Activation keys, invoice data, and PII returned in API responses are transmitted in cleartext over HTTP, readable by any network observer.	TLS everywhere; mark all sensitive response fields as non-cacheable (Cache-Control: no-store, no-cache); use secure, HttpOnly, SameSite cookies for any session identifiers; avoid putting sensitive data in URL query parameters (logged by proxies and servers).
62	HTTP flood / slow-loris attack	Denial of service	High	Open		Attacker opens thousands of slow HTTP connections to the Spring Boot server, holding threads open without completing requests, exhausting the server thread pool and blocking legitimate traffic.	Deploy behind a reverse proxy (Nginx) that handles connection timeouts; set maximum request body size and read timeout; use a WAF to detect and block HTTP flood patterns; Nginx's limit_conn and limit_req modules; configure Spring Boot's server.connection-timeout.

Authentication Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
57	Token header/payload tampering	Tampering	Medium	Open		Attacker intercepts a JWT, modifies the role claim in the payload from BUYER to ADMIN, and re-signs with a known weak secret (HS256 with secret "secret").	Use asymmetric signing (RS256) from Keycloak; backend validates signature against JWKS endpoint; reject tokens signed with HS256 or weak keys; validate all standard claims (iss, aud, exp, nbf).
58	Token interception / exposure in logs	Information disclosure	High	Open		Bearer token is included in a GET request URL as a query parameter (?token=...) which is logged by Nginx access logs, a CDN, and browser history — exposing a long-lived token.	Always transmit JWTs in the Authorization header, never in URLs; set short expiry on access tokens (≤15 min); use refresh token rotation; purge tokens from all log pipelines; monitor for token reuse from unexpected IPs.
59	Token validation overhead / JWKS endpoint unavailability	Denial of service	Medium	Open		If Spring Security fetches the JWKS endpoint on every request to validate token signatures, a Keycloak outage causes all API requests to fail. Alternatively, an attacker floods the JWKS endpoint.	Cache the JWKS public key locally with a reasonable TTL (e.g. 1 hour); implement offline token validation fallback; deploy Keycloak in HA configuration behind a load balancer; health-check the JWKS endpoint in the application startup; use circuit breaker for JWKS fetching.

Authentication Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
57	Token header/payload tampering	Tampering	Medium	Open		Attacker intercepts a JWT, modifies the role claim in the payload from BUYER to ADMIN, and re-signs with a known weak secret (HS256 with secret "secret").	Use asymmetric signing (RS256) from Keycloak; backend validates signature against JWKS endpoint; reject tokens signed with HS256 or weak keys; validate all standard claims (iss, aud, exp, nbf).
58	Token interception / exposure in logs	Information disclosure	High	Open		Bearer token is included in a GET request URL as a query parameter (?token=...) which is logged by Nginx access logs, a CDN, and browser history — exposing a long-lived token.	Always transmit JWTs in the Authorization header, never in URLs; set short expiry on access tokens (≤15 min); use refresh token rotation; purge tokens from all log pipelines; monitor for token reuse from unexpected IPs.
59	Token validation overhead / JWKS endpoint unavailability	Denial of service	Medium	Open		If Spring Security fetches the JWKS endpoint on every request to validate token signatures, a Keycloak outage causes all API requests to fail. Alternatively, an attacker floods the JWKS endpoint.	Cache the JWKS public key locally with a reasonable TTL (e.g. 1 hour); implement offline token validation fallback; deploy Keycloak in HA configuration behind a load balancer; health-check the JWKS endpoint in the application startup; use circuit breaker for JWKS fetching.

User Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

User Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Order Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Order Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

User Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

User Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Data Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Game Data Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Game Image Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Invoice PDF content manipulation in transit	Tampering	Medium	Open		On an unencrypted internal network, an attacker on the same host intercepts the byte stream between the PDF generation library and the file-write operation, replacing invoice amount fields.	Generate and write invoice PDFs atomically within the application process (no separate file service); use TLS even for internal service communication; verify PDF integrity with a hash stored in the DB after generation.
70	Game image/invoice served without access control	Information disclosure	High	Open		Invoice PDFs are served from GET /files/invoices/{filename}. Without auth checks on this endpoint, any user can enumerate filenames and download invoices belonging to other users.	Authenticate and authorise all file delivery endpoints; verify that the requesting user's ID matches the invoice owner before streaming the file; use random UUID filenames (not predictable sequences); consider pre-signed, time-limited URLs for download links.
71	Concurrent invoice PDF generation overloading CPU/memory	Denial of service	Medium	Open		Attacker or bot places and completes 100 orders simultaneously. Each triggers synchronous PDF invoice generation (CPU and memory intensive), causing a Java heap OOM or CPU spike that degrades the entire application.	Move PDF generation to an asynchronous background worker (message queue, e.g. RabbitMQ or Spring @Async); limit concurrent PDF generation tasks; apply per-user order rate limiting; set JVM heap limits appropriately in Docker container resource constraints.

Game Image Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Invoice PDF content manipulation in transit	Tampering	Medium	Open		On an unencrypted internal network, an attacker on the same host intercepts the byte stream between the PDF generation library and the file-write operation, replacing invoice amount fields.	Generate and write invoice PDFs atomically within the application process (no separate file service); use TLS even for internal service communication; verify PDF integrity with a hash stored in the DB after generation.
70	Game image/invoice served without access control	Information disclosure	High	Open		Invoice PDFs are served from GET /files/invoices/{filename}. Without auth checks on this endpoint, any user can enumerate filenames and download invoices belonging to other users.	Authenticate and authorise all file delivery endpoints; verify that the requesting user's ID matches the invoice owner before streaming the file; use random UUID filenames (not predictable sequences); consider pre-signed, time-limited URLs for download links.
71	Concurrent invoice PDF generation overloading CPU/memory	Denial of service	Medium	Open		Attacker or bot places and completes 100 orders simultaneously. Each triggers synchronous PDF invoice generation (CPU and memory intensive), causing a Java heap OOM or CPU spike that degrades the entire application.	Move PDF generation to an asynchronous background worker (message queue, e.g. RabbitMQ or Spring @Async); limit concurrent PDF generation tasks; apply per-user order rate limiting; set JVM heap limits appropriately in Docker container resource constraints.

Invoice Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Invoice PDF content manipulation in transit	Tampering	Medium	Open		On an unencrypted internal network, an attacker on the same host intercepts the byte stream between the PDF generation library and the file-write operation, replacing invoice amount fields.	Generate and write invoice PDFs atomically within the application process (no separate file service); use TLS even for internal service communication; verify PDF integrity with a hash stored in the DB after generation.
70	Game image/invoice served without access control	Information disclosure	High	Open		Invoice PDFs are served from GET /files/invoices/{filename}. Without auth checks on this endpoint, any user can enumerate filenames and download invoices belonging to other users.	Authenticate and authorise all file delivery endpoints; verify that the requesting user's ID matches the invoice owner before streaming the file; use random UUID filenames (not predictable sequences); consider pre-signed, time-limited URLs for download links.
71	Concurrent invoice PDF generation overloading CPU/memory	Denial of service	Medium	Open		Attacker or bot places and completes 100 orders simultaneously. Each triggers synchronous PDF invoice generation (CPU and memory intensive), causing a Java heap OOM or CPU spike that degrades the entire application.	Move PDF generation to an asynchronous background worker (message queue, e.g. RabbitMQ or Spring @Async); limit concurrent PDF generation tasks; apply per-user order rate limiting; set JVM heap limits appropriately in Docker container resource constraints.

Invoice Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
69	Invoice PDF content manipulation in transit	Tampering	Medium	Open		On an unencrypted internal network, an attacker on the same host intercepts the byte stream between the PDF generation library and the file-write operation, replacing invoice amount fields.	Generate and write invoice PDFs atomically within the application process (no separate file service); use TLS even for internal service communication; verify PDF integrity with a hash stored in the DB after generation.
70	Game image/invoice served without access control	Information disclosure	High	Open		Invoice PDFs are served from GET /files/invoices/{filename}. Without auth checks on this endpoint, any user can enumerate filenames and download invoices belonging to other users.	Authenticate and authorise all file delivery endpoints; verify that the requesting user's ID matches the invoice owner before streaming the file; use random UUID filenames (not predictable sequences); consider pre-signed, time-limited URLs for download links.
71	Concurrent invoice PDF generation overloading CPU/memory	Denial of service	Medium	Open		Attacker or bot places and completes 100 orders simultaneously. Each triggers synchronous PDF invoice generation (CPU and memory intensive), causing a Java heap OOM or CPU spike that degrades the entire application.	Move PDF generation to an asynchronous background worker (message queue, e.g. RabbitMQ or Spring @Async); limit concurrent PDF generation tasks; apply per-user order rate limiting; set JVM heap limits appropriately in Docker container resource constraints.

Insert/Query Game Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Insert/Query Order Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Insert/Query User Library Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Insert/Query User Information Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Database Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

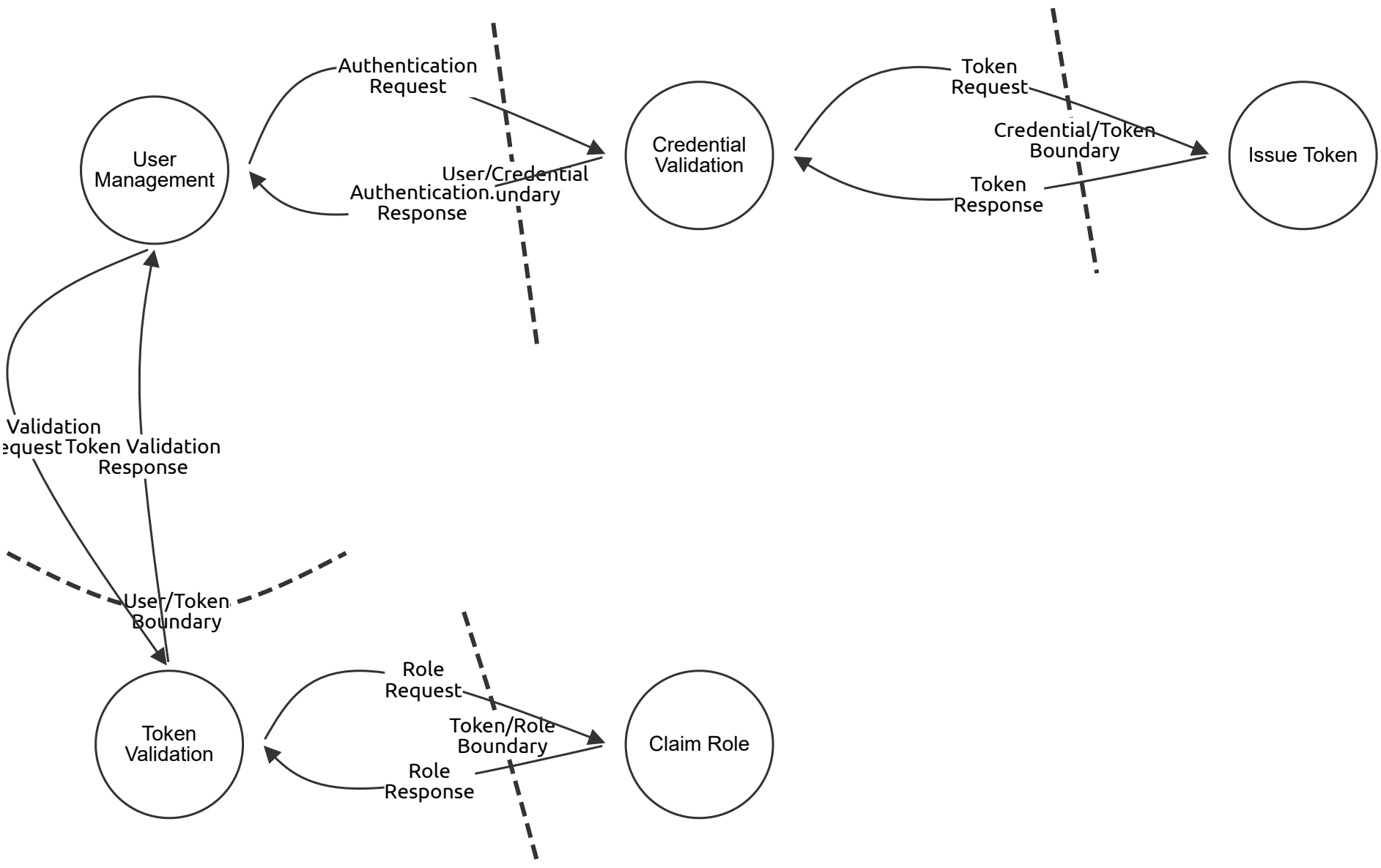
Database (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
39	SQL injection / ORM injection	Tampering	Critical	Open		A malformed game search query bypasses the JPA parameterisation (e.g. via a native query built with string concatenation) and executes DROP TABLE orders; in the database.	Use only parameterised queries and JPA named parameters; never use string concatenation in native queries; apply principle of least privilege to the DB user (no DROP/DDL rights in application account); run Flyway migrations with a separate privileged user.
40	No DB-level audit trail	Repudiation	High	Open		A record in the orders table is modified directly (bypassing the application layer, e.g. by a DBA). Without DB-level change logging, the modification cannot be attributed or detected.	Enable PostgreSQL pgaudit extension to log all DDL and DML; configure row-level security where appropriate; use Flyway for all schema changes; restrict direct DB access to emergency procedures with full logging; consider application-level audit tables with insert-only access.
41	Unencrypted PII and activation keys at rest	Information disclosure	High	Open		The PostgreSQL data directory is stored unencrypted. A cloud storage misconfiguration or disk theft exposes all user PII, hashed passwords, and plaintext activation keys.	Enable PostgreSQL Transparent Data Encryption or use encrypted AWS RDS/EBS volumes; encrypt sensitive columns (activation keys, payment references) at the application layer using AES-256 before storing; hash passwords with bcrypt (already handled by Spring Security / Keycloak); restrict network access to DB to application subnet only.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
42	Connection pool exhaustion / long-running queries	Denial of service	High	Open		An attacker triggers a complex game search with many filters, generating a full-table-scan query that holds DB connections for minutes, starving other requests of connections from HikariCP pool.	Set query timeouts in Spring Data / JPA (spring.transaction.default-timeout); add DB indices on search columns (game title, category, publisher_id); configure HikariCP connection pool limits and timeouts; use read replicas for search/reporting queries; implement circuit breaker for DB calls.

DFD - Level 1 - Auth API

Auth API DFD



DFD - Level 1 - Auth API

User Management (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Credential Validation (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Issue Token (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Token Validation (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Claim Role (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Authentication Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Authentication. Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Token Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Token Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Token Validation Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Token Validation Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Role Request (Data Flow)

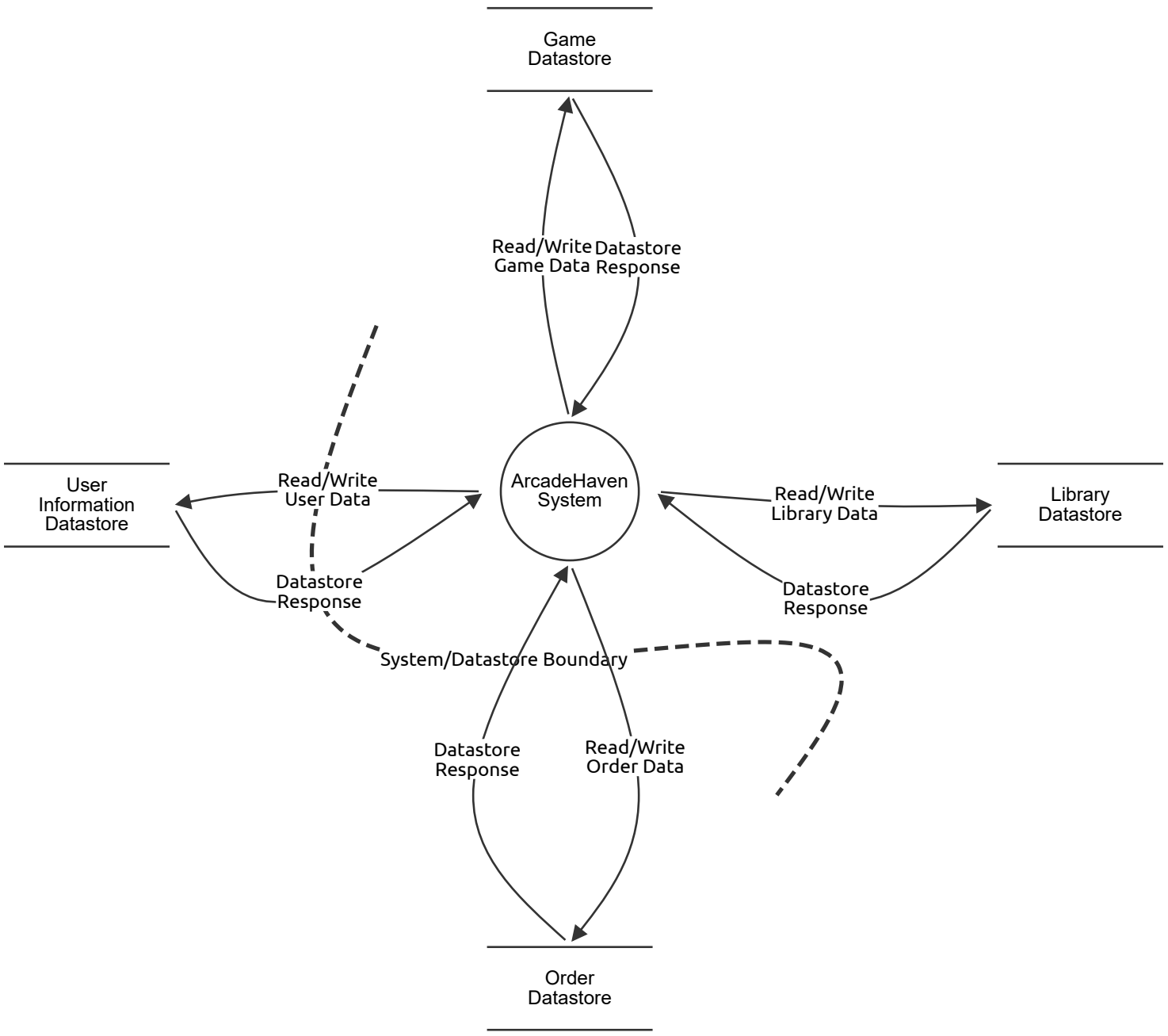
Number	Title	Type	Severity	Status	Score	Description	Mitigations

Role Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

DFD - Level 1 - Database

Database interactions DFD



DFD - Level 1 - Database

ArcadeHaven System (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Datastore (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Library Datastore (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Order Datastore (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

User Information Datastore (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Read/Write Game Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Datastore Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Read/Write Library Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Datastore Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Read/Write Order Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Datastore Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Read/Write User Data (Data Flow)

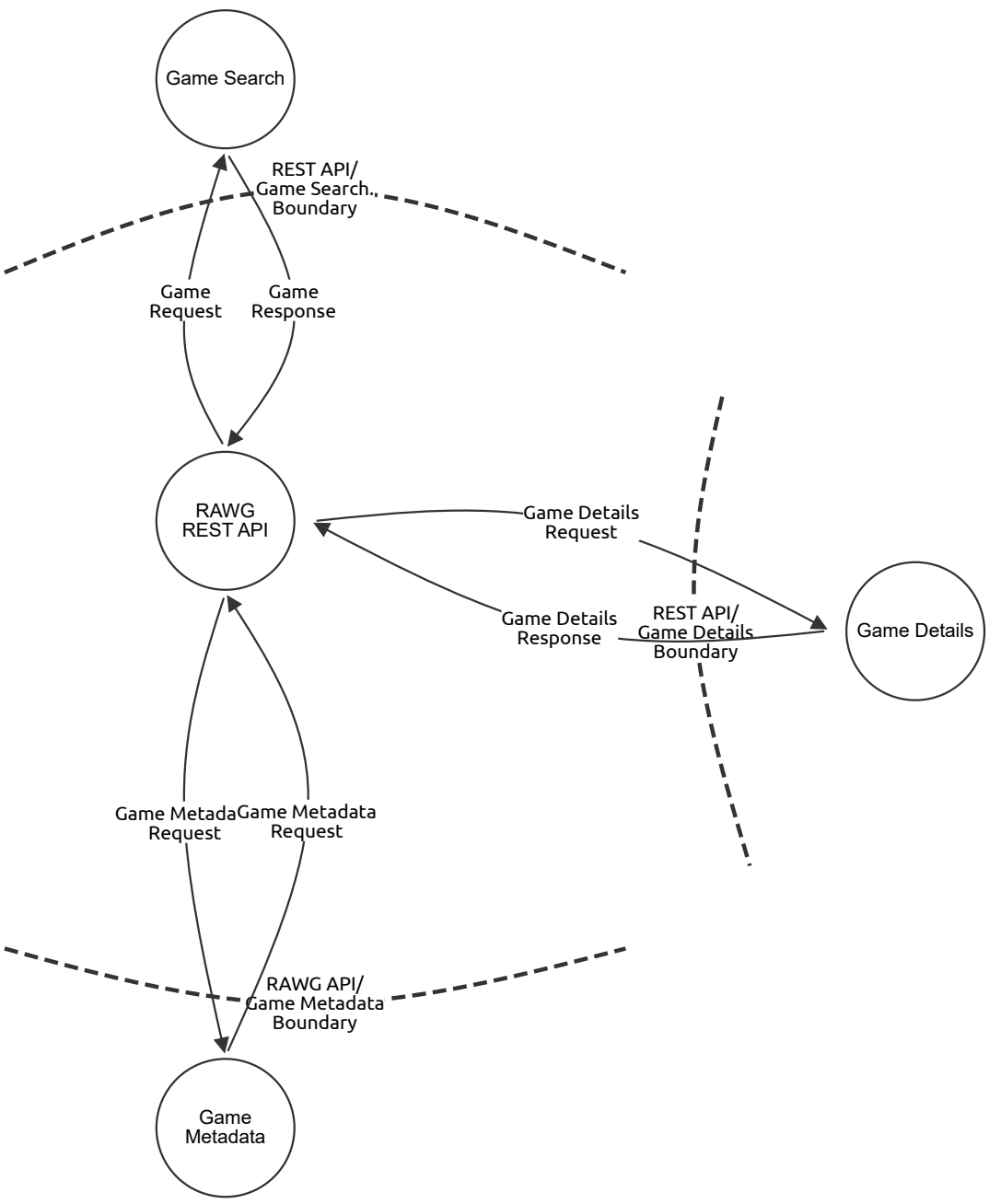
Number	Title	Type	Severity	Status	Score	Description	Mitigations

Datastore Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

DFD - Level 1 - RAWG API

RAWG API DFD



DFD - Level 1 - RAWG API

Game Search (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

RAWG REST API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Details (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Details Request (Data Flow)

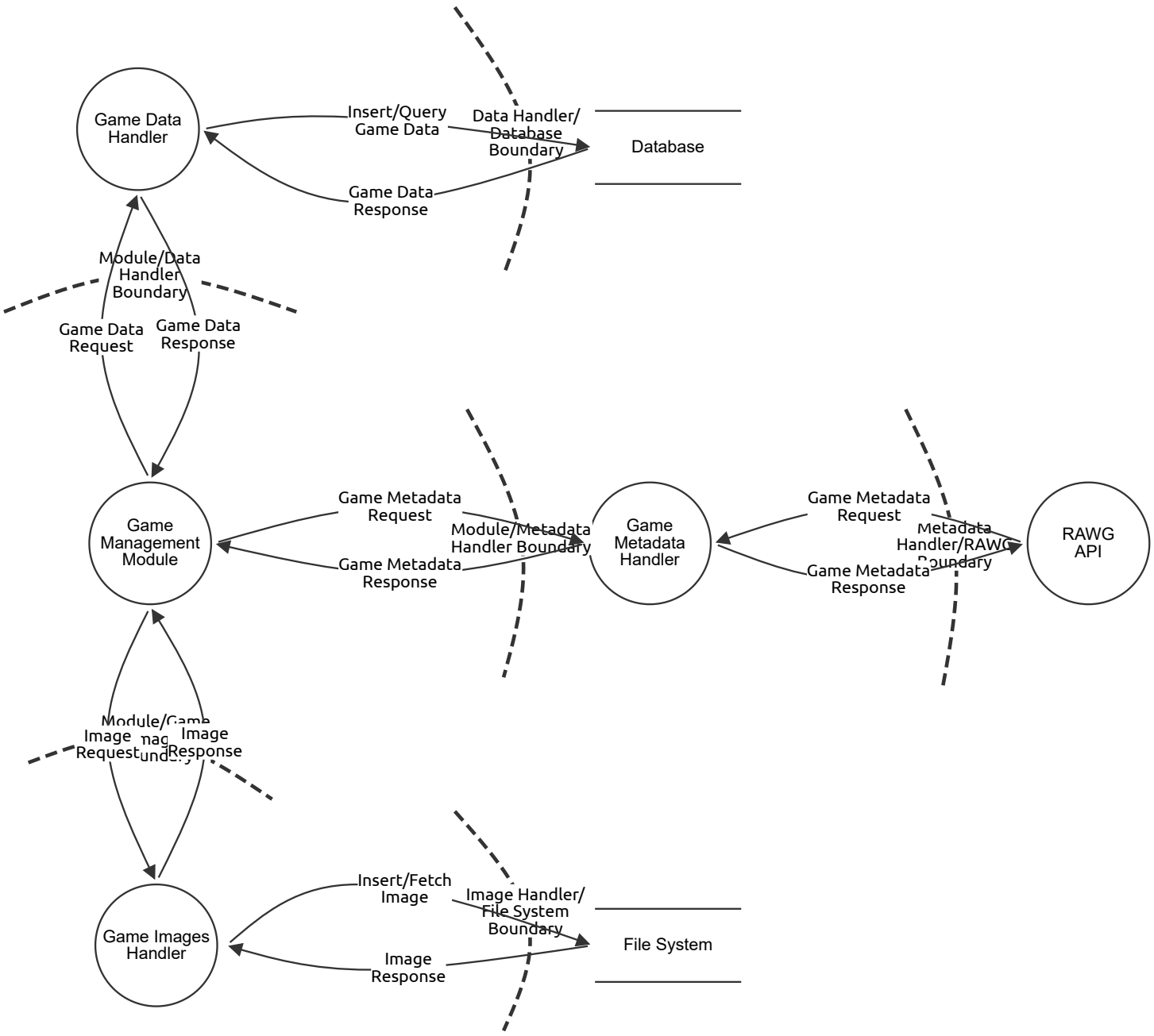
Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Details Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

DFD - Level 2 - Game Management

ArcadeHaven DFD - Level 2 - Game Management



DFD - Level 2 - Game Management

Game Data Handler (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Database (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Management Module (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Handler (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

RAWG API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Images Handler (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

File System (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Data Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.

Number	Title	Type	Severity	Status	Score	Description	Mitigations
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Game Data Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Insert/Query Game Data (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
63	SQL injection via unsanitised flow payload	Tampering	High	Open		An unsanitised game search term is interpolated directly into a native JPA query: SELECT * FROM games WHERE title = '${input}'. Input of ' OR '1'='1 returns all records.	Use Spring Data JPA repository methods or @Query with :param bindings exclusively; ban native query string concatenation via code review and SonarQube rules; parameterise all dynamic filters.
64	Credentials in DB connection string exposed	Information disclosure	Critical	Open		The application.properties file containing spring.datasource.password=plaintext is committed to the GitHub repository, exposing DB credentials to anyone with repo access.	Store DB credentials in environment variables or a secrets manager (e.g. Docker secrets, HashiCorp Vault, or GitHub Actions secrets); never commit credentials to source control; add .gitignore rules and pre-commit hooks scanning for secrets (e.g. truffleHog, detect-secrets).
65	Unbounded query results exhausting memory	Denial of service	Medium	Open		GET /games with no pagination causes a SELECT * FROM games with no LIMIT, loading the entire catalogue into the JPA entity list in memory. As the catalogue grows, this causes OOM on the application server.	Enforce pagination via Spring Data Pageable on all repository methods that return collections; set a maximum page size (e.g. 100); add DB indices on all ORDER BY and WHERE columns used in catalogue queries; query timeout configuration in HikariCP.

Game Data Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
66	Malicious content in RAWG response stored to DB	Tampering	Medium	Open		A compromised or spoofed RAWG response contains a game description with JavaScript. The string is stored in the DB and later rendered as raw HTML in the frontend, resulting in stored XSS.	Sanitise all RAWG-sourced string fields before persistence; treat all external data as untrusted; apply HTML encoding at render time; enforce Content-Security-Policy with script-src 'self' to prevent XSS execution.
67	RAWG API key exposed in logs or source	Information disclosure	Medium	Open		The RAWG API key is hardcoded in a GameMetadataHandler.java constant. It appears in SonarQube scan output, GitHub commits, or application logs, allowing third parties to use the key.	Store API keys in environment variables or a secrets manager; exclude from source control; rotate API keys regularly; monitor for unexpected RAWG API usage (rate limit alerts); use application-level secret scanning in CI (truffleHog, GitGuardian).
68	RAWG rate limiting / external dependency failure	Denial of service	Medium	Open		The RAWG free tier has a rate limit. A Publisher bulk-imports 500 games in one operation, exhausting the daily API quota and breaking the Game Management module for all subsequent users.	Implement client-side rate limiting and request queuing for RAWG calls; cache RAWG responses locally (Redis or DB) with appropriate TTL; degrade gracefully when RAWG is unavailable (show cached data, disable metadata enrichment); circuit breaker pattern with fallback.

Game Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Game Metadata Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Image Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Image Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Insert/Fetch Image (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations

Image Response (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations